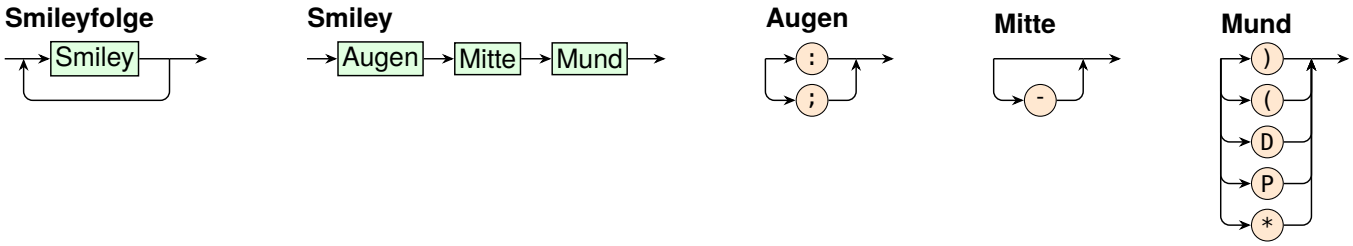
1.3 Syntaxdiagramme

In 1.2 hast du gemerkt: In Worten lassen sich Smileys nicht eindeutig beschreiben. Die folgenden fünf Diagramme legen dagegen *genau* fest, welche Wörter über $A_3 = \{\texttt{:}, \texttt{;}, \texttt{-}, \texttt{)}, \texttt{(}, \texttt{D}, \texttt{P}, \texttt{*}\}$ Smileyfolgen sind.



Aufgabe 1 *Diagramme lesen*

Lies jedes Diagramm wie ein **Gleisnetz**: Start bei **Smileyfolge**, folge den Pfeilen, an jeder Weiche darfst du wählen. Ein **rundes** Feld schreibt sein Zeichen ins Wort, bei einem **eckigen** fährst du das Diagramm dieses Namens ab und kehrst zurück.

a) Kreuze an, welche Wörter gültige Smileyfolgen sind. Begründe bei den ungültigen.

`:-)`	☒		`:-`	☐	Mund fehlt
`;P:-(`	☒		`:--D`	☐	Mitte erlaubt nur eine Nase
`;*`	☒		`-)`	☐	beginnt nicht mit Augen

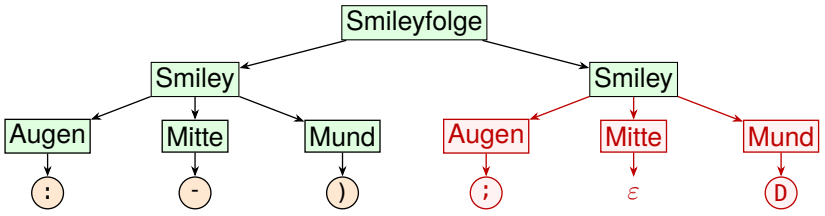
b) Ordne die Bausteine *Aneinanderreihung, Alternative, Option, Wiederholung* zu. Wirkung?

Diagramm	Baustein	Wirkung
Smileyfolge	Wiederholung	Smiley kommt **mindestens einmal**, beliebig oft
Smiley	Aneinanderreihung	Augen, Mitte, Mund genau in dieser Reihenfolge
Augen, Mund	Alternative	**genau einer** der Zweige wird gewählt
Mitte	Option	`-` kommt **einmal oder keinmal** vor

Grammatik und Syntaxdiagramm

Eine formale Sprache wird durch eine **Grammatik** eindeutig festgelegt. Sie besteht aus einem **Startsymbol** (hier: Smileyfolge) und **Produktionsregeln** (kurz: Regeln), nach denen daraus gültige Wörter gebildet (*produziert*) werden. Die Regeln enthalten **Terminale** (Zeichen des Alphabets, im Diagramm rund, stehen im Wort) und **Nichtterminale** (Namen anderer Regeln, im Diagramm eckig, stehen nie im Wort). Eine Darstellungsform der Regeln ist das **Syntaxdiagramm**.

Aufgabe 2 *Ableitungsbaum*



Der **Ableitungsbaum** zeigt, wie das Wort `:-);D` aus dem Startsymbol entsteht.

a) Vervollständige den Baum.

b) Lies die Blätter von links nach rechts. Was erhältst du?
das Wort `:-);D`

c) Warum steht unter einer „Mitte“ ε?
Option ohne `-` gewählt

Aus dem Startsymbol werden Nichtterminale so lange nach ihren Produktionsregeln ersetzt, bis nur noch Terminale übrig sind: Das Wort wird **abgeleitet** (produziert). Im Ableitungsbaum steht das Startsymbol in der Wurzel, die Terminale stehen in den Blättern.

Aufgabe 3 *Selbst bauen: RGB-Farbcodes*

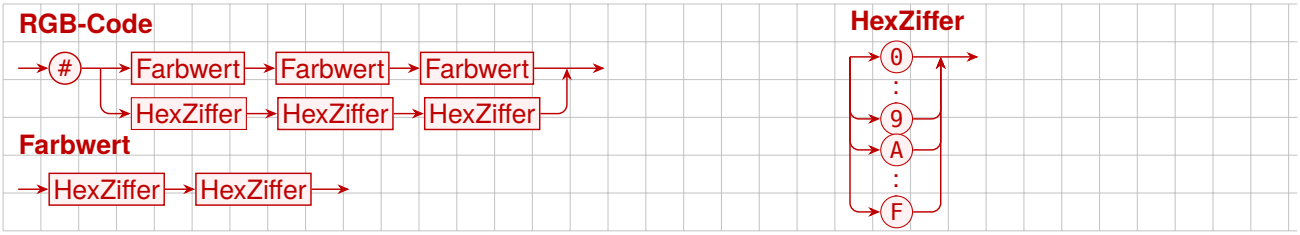
So gehst du vor: ① Beispiele zerlegen ② gleiche Teile benennen (je eine Regel) ③ Bausteine wählen ④ testen

a) Zerlege den Farbcode `#F3BC0D` (1.1, Aufgabe 6) in gleich aufgebaute Teile und benenne sie.

`#` | `F3` | `BC` | `0D`: dreimal **Farbwert** aus zwei **HexZiffern**

b) Zeichne die Diagramme **RGB-Code**, **Farbwert** und **HexZiffer**. Teste mit `#9900BB`, `#9900B`, `9900BB`.

c) Erweitere **RGB-Code** um die Kurzform `#F3B` (= `#FF33BB`). **d)** (mündlich) Was ist gleich, was nicht?



Aufgabe 4 *Selbst bauen: Autokennzeichen*

Vereinfacht: **1 bis 3 Buchstaben** für den Bezirk (`M`, `TS`, `BGL`), ein Bindestrich, **1 oder 2 Buchstaben**, ein Leerzeichen ␣ und eine **1- bis 4-stellige Zahl ohne führende Null**.

a) Korrekt (k), syntaktisch falsch (sy) oder semantisch falsch (se)? Begründe kurz.

`TS-NA 48484` sy: 5 Ziffern

`P1-JO 42` sy: Ziffer im Bezirk

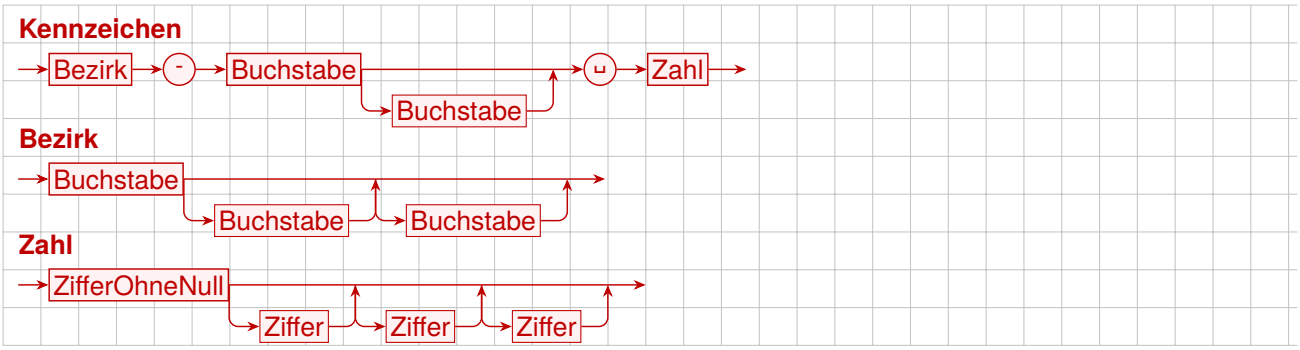
`TOR-TE 3` se: Bezirk TOR gibt es nicht

`RO-T 2209` k

`IN-FO 13` k

`ED-DY 007` sy: führende Null

b) Zeichne **Kennzeichen**, **Bezirk**, **Zahl**. Gegeben: *Buchstabe* (`A`…`Z`), *Ziffer* (`0`…`9`), *ZifferOhneNull* (`1`…`9`).



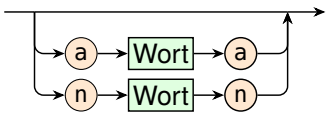
c) Eine Wiederholung erlaubt beliebig viele Durchläufe.

Für „1 bis 3“ nimmt man zwei Optionen hintereinander

Führende Nullen verhindert man so: Die erste Ziffer ist eine ZifferOhneNull.

Aufgabe 5 *Rekursive Regeln*

Wort



Das Diagramm **Wort** verweist auf sich selbst. Gib an:

a) Alphabet: {a, n}

b) Vier Wörter: ε, aa, anna, nannan

c) Beschreibe die Sprache: Palindrome gerader Länge

d) Warum geht das nicht mit einer Wiederholung?

Hinten muss das Gegenstück zu vorne stehen.

Rekursive Regeln

Eine Regel ist **rekursiv**, wenn ein Nichtterminal (direkt oder indirekt) auf sich selbst verweist. Endlich viele Regeln erzeugen so unendlich viele Wörter; ein Weg ohne Selbstaufruf beendet die Ableitung.